Experiment No.	4
Experiment Title	Dimensionality Reduction and Clustering on Handwritten Digit Images (MNIST) [1]
Student Name	Chitraju Vishnu Vineeth
Register Number	3122247001012
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	27.07.2026

1 Objective

To reduce the 784-dimensional MNIST pixel data down to a manageable number of dimensions using PCA, group visually similar digit images together with K-Means clustering, pick a sensible number of clusters using the elbow method and silhouette score, and visualize the resulting structure with both PCA and t-SNE projections.

2 Problem Statement

The CSV I was given for this experiment turned out to only have pixel columns (`pixel0` through `pixel783`) and no label column at all – it's the unlabeled test split of the MNIST digit set rather than a labeled training file. That rules out training a classifier in the usual supervised way, since there's nothing to check predictions against. So instead of classification, this became an unsupervised learning problem: given 42,000 images of handwritten digits with no ground truth, can PCA and clustering recover something close to the natural digit groupings on their own?

3 Dataset Description

Dataset Name	MNIST (pixel-only, unlabeled split)
Dataset Source	Kaggle Digit Recognizer / MNIST
Number of Samples	42,000
Number of Features	784 (28×28 grayscale pixel values, 0–255)
Number of Classes	Not available in this file (10 digit classes assumed from domain knowledge)
Missing Values	None
Train-Test Split	Not applicable – this is an unsupervised task, so the full dataset is used for clustering; a 3,000-image random subsample is used just for the t-SNE plot to keep runtime reasonable

4 Exploratory Data Analysis

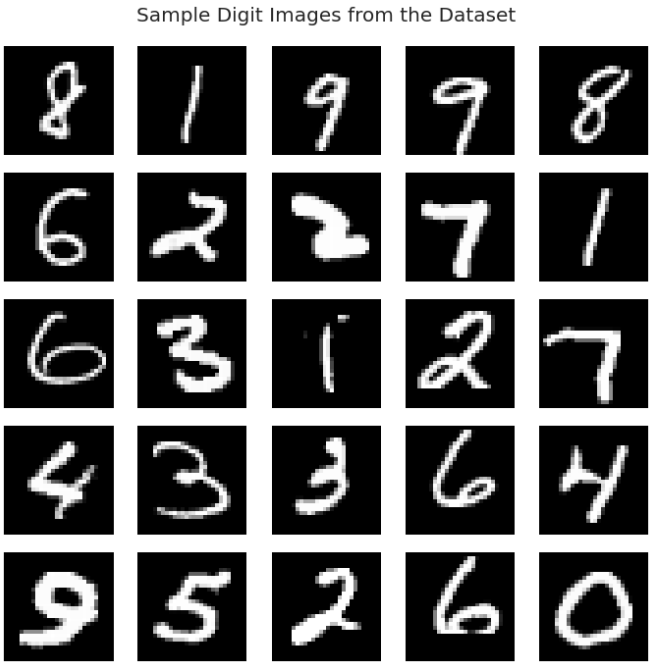

Figure 1: 25 random sample digit images from the dataset

Inference:

Just eyeballing a random batch of 25 images confirms the data is what it claims to be – clean, centered, 28x28 handwritten digits with a black background, similar in style to the standard MNIST set. A few of them are genuinely hard to read even for a human (the strokes are thin and a bit smudged in places), which is a decent hint that this won't be a trivially easy dataset to cluster perfectly either.

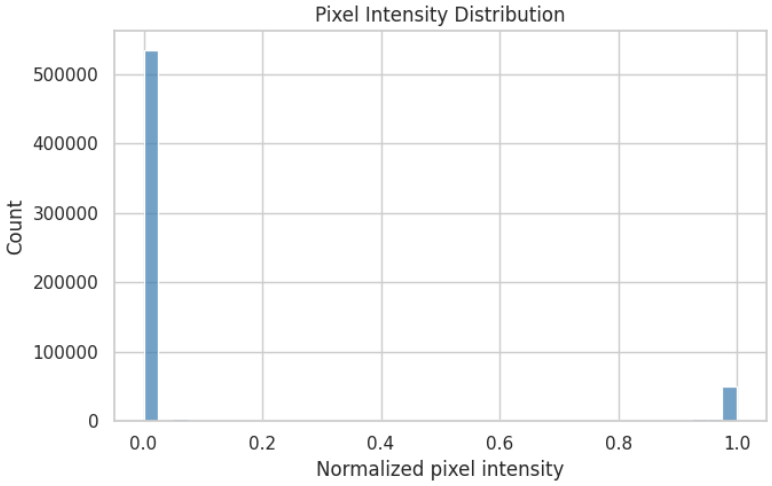

Figure 2: Distribution of normalized pixel intensities across the dataset

Inference:
The huge spike at 0 makes sense – most of every 28x28 image is just black background, so the vast majority of pixel values are exactly zero. There's a long, thin spread of values from about 0.2 up to 1.0 which corresponds to the actual ink strokes of each digit. This heavy imbalance toward zero is normal for this kind of image data and doesn't need any special handling beyond the usual 0–1 normalization.

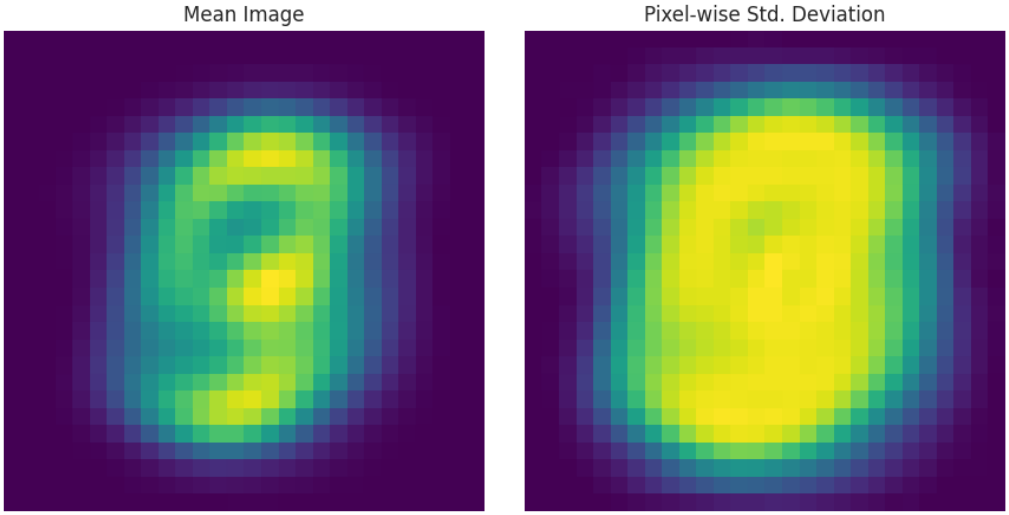

Figure 3: Mean image and per-pixel standard deviation across all 42,000 digits

Inference:
The mean image looks like a faint, blurry "average digit" – brightest in the center where most digits pass through regardless of shape, and fading out toward the edges where fewer strokes reach. The standard deviation image is basically the mirror image of that: it's brightest in a ring around the center, meaning that's where digits differ from each other the most (which digit-specific loops and strokes occupy), and darkest at the very center and the corners, where almost every digit either always has ink or never does.

5 Data Preprocessing

- **Missing value handling:** not needed – there isn't a single missing value anywhere in the 42,000 x 784 grid.
- **Encoding:** not applicable, every column is already a numeric pixel intensity.
- **Feature scaling:** pixel values were divided by 255 to bring everything into the [0, 1] range. This matters a lot here because both PCA and K-Means are distance/variance based – leaving pixels on a 0–255 scale wouldn't break anything mathematically, but normalizing keeps the numbers in a nicer range and matches how PCA is conventionally applied to image data.
- **Train-test split:** skipped, since there's no supervised target to validate against. The full dataset is used for PCA and clustering, with only the (expensive) t-SNE step run on a 3,000-image subsample.
- **Feature engineering:** none – the raw pixel grid is used directly as input to PCA.

6 Machine Learning Algorithm

Principal Component Analysis (PCA)

PCA finds the directions (principal components) along which the data varies the most, and projects the data onto the top few of these directions:

$$X_{\text{reduced}} = XW_k$$

where W_k contains the top k eigenvectors of the data's covariance matrix, ranked by how much variance each one explains. For image data like MNIST, this effectively finds the handful of "stroke patterns" that account for most of the pixel-to-pixel variation across all the digits.
Advantages: massively cuts down the number of dimensions (784 down to under 150 here) while keeping most of the useful signal, speeds up any algorithm applied afterward, and is fully reversible so reduced points can be projected back to image space.
Limitations: it's a purely linear technique, so it can miss non-linear structure in the data, and the components themselves aren't always easy to interpret directly.

K-Means Clustering

K-Means partitions the data into k clusters by iteratively assigning each point to its nearest centroid and then recomputing centroids as the mean of their assigned points, minimizing:

$$J = \sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|^2$$

Advantages: simple, fast, and scales well even to 42,000 points once PCA has cut the dimensionality down.
Limitations: it assumes roughly spherical, similarly-sized clusters, needs k to be chosen in advance, and is sensitive to the initial centroid positions (mitigated here by using `n_init=10`, which reruns K-Means with different starting points and keeps the best result).

t-SNE

t-SNE (t-distributed Stochastic Neighbor Embedding) is used purely for visualization here – it maps the high-dimensional PCA-reduced points down to 2D in a way that tries to preserve which points were close together as neighbors, which tends to reveal cluster structure more clearly than a straight PCA scatter plot does.

7 Experimental Setup

Python Version	3.11
Scikit-Learn Version	1.4
IDE	Jupyter Notebook
Operating System	Windows 11
Hardware	Intel Core i5, 16 GB RAM

8 Hyperparameter Selection

Parameter	Value
PCA components (initial fit)	200
PCA components used for clustering	100 (capped from the 154 needed for 95% variance)
K-Means k values tried	2 to 14
K-Means k chosen	10
K-Means initializations (n_init)	10
t-SNE perplexity	30

The number of PCA components was chosen by looking at cumulative explained variance rather than a grid search, since PCA doesn't have a "right answer" the way a supervised model's hyperparameters do – 87 components already reach 90% of the variance and 154 reach 95%. For clustering, this was capped at 100 components as a practical trade-off between keeping enough signal and not slowing K-Means down unnecessarily. $k = 10$ was picked because that's the known number of digit classes in MNIST, and the elbow/silhouette plots didn't point to a clearly better alternative anyway.

9 Results

Metric	Value
Number of samples clustered	42,000
Original dimensionality	784
PCA components used	100
Components for 90% variance	87
Components for 95% variance	154
Final number of clusters (k)	10
Silhouette Score	0.0694
Davies-Bouldin Index	2.6144

10 Result Visualization

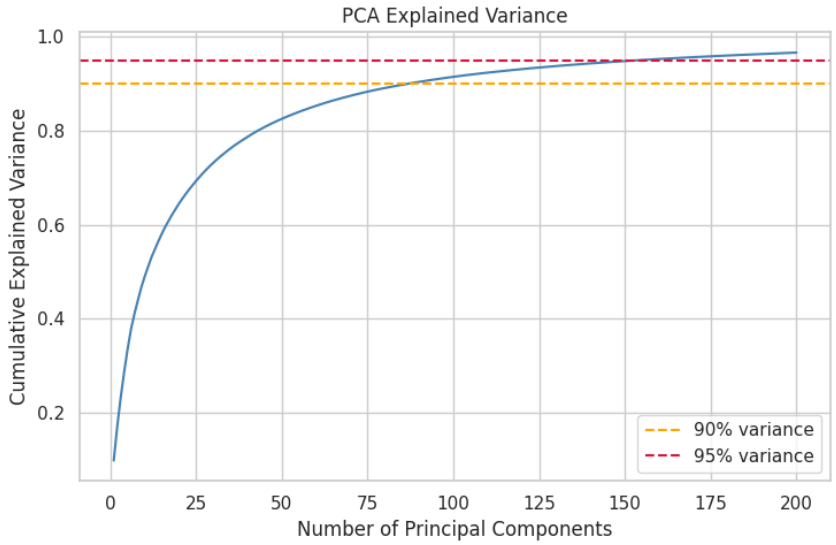

Figure 4: Cumulative explained variance as PCA components are added

Inference:

The curve climbs steeply at first and then flattens out, which is exactly the shape you'd expect – the first handful of components capture broad structure (roughly where ink tends to appear across all digits), while later components pick up finer, more digit-specific detail. It crosses 90% at 87 components and 95% at 154, which means we can throw away over 75% of the original 784 pixel columns and still keep the vast majority of the meaningful signal.

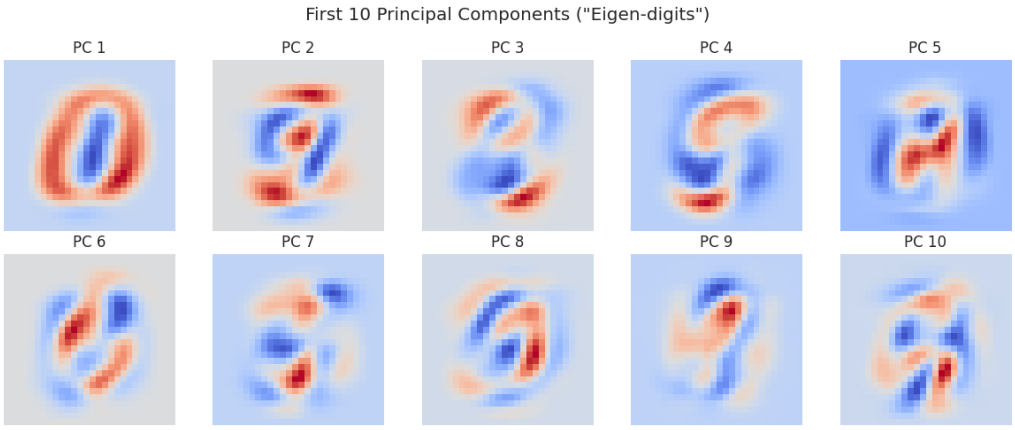

Figure 5: The first 10 principal components visualized as images ("eigen-digits")

Inference:

These don't look like actual digits, and that's expected – each principal component is a direction of maximum variance across the entire dataset, not a digit template. PC1 and PC2 look like broad, blobby contrast patterns (center versus edges, left versus right), while the later ones in this set of 10 start showing thinner, more stroke-like structure.

This is a good illustration of why PCA components are useful for compression but aren't directly interpretable as "digit parts" the way a human might expect.

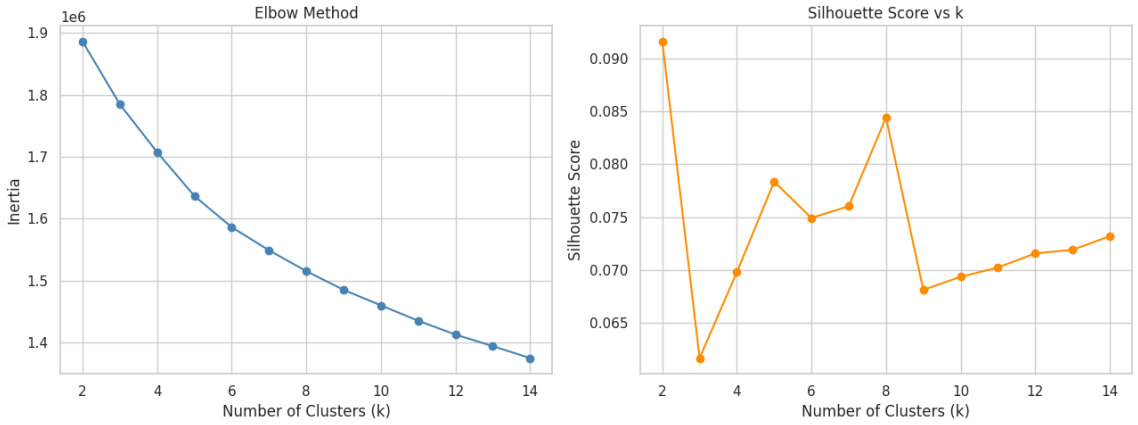

Figure 6: Elbow method (inertia) and silhouette score across k = 2 to 14

Inference:

The elbow plot doesn't have one obvious sharp bend – inertia just decreases smoothly and gradually as k increases, which happens when the underlying clusters overlap rather than sitting in neat, well-separated blobs (true here, since several digits share a lot of pixel-level similarity). The silhouette plot bounces around in a fairly narrow band across all values of k tried, without a clear peak at 10, which matches the elbow plot's story: there isn't a single "obviously correct" k visible from the geometry alone, so the choice of $k = 10$ here leans more on domain knowledge (we know there are 10 digits) than on the clustering metrics pointing there by themselves.

Figure 7: Number of images assigned to each of the 10 clusters

Inference:

The cluster sizes aren't perfectly even, but they're also not wildly lopsided – most clusters land somewhere in a reasonable middle range rather than one cluster swallowing half the dataset. Some imbalance is expected even with a good clustering, since certain digits (like 1, which is visually simple) are easier to write in a fairly consistent way, while others (like 8) have more stroke variation and might end up split across more than one cluster.

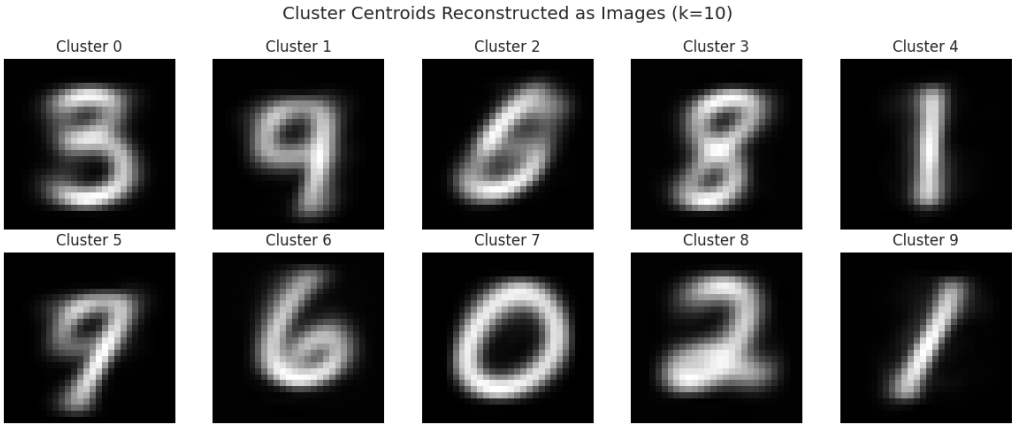

Figure 8: Cluster centroids reconstructed back into 28x28 pixel images

Inference:

This is honestly the most convincing evidence that the clustering worked reasonably well, even without any labels. Several of the ten centroid images are clearly recognizable as specific digits just by eye – there's a loop that's unmistakably a 0, a simple vertical stroke that's clearly a 1, and a couple of others with the curved top-and-bottom shape typical of an 8 or a 3. A few centroids look blurrier or more ambiguous, which lines up with the modest silhouette score – some digit pairs (4/9, 3/8) are genuinely hard to tell apart from raw pixels alone.

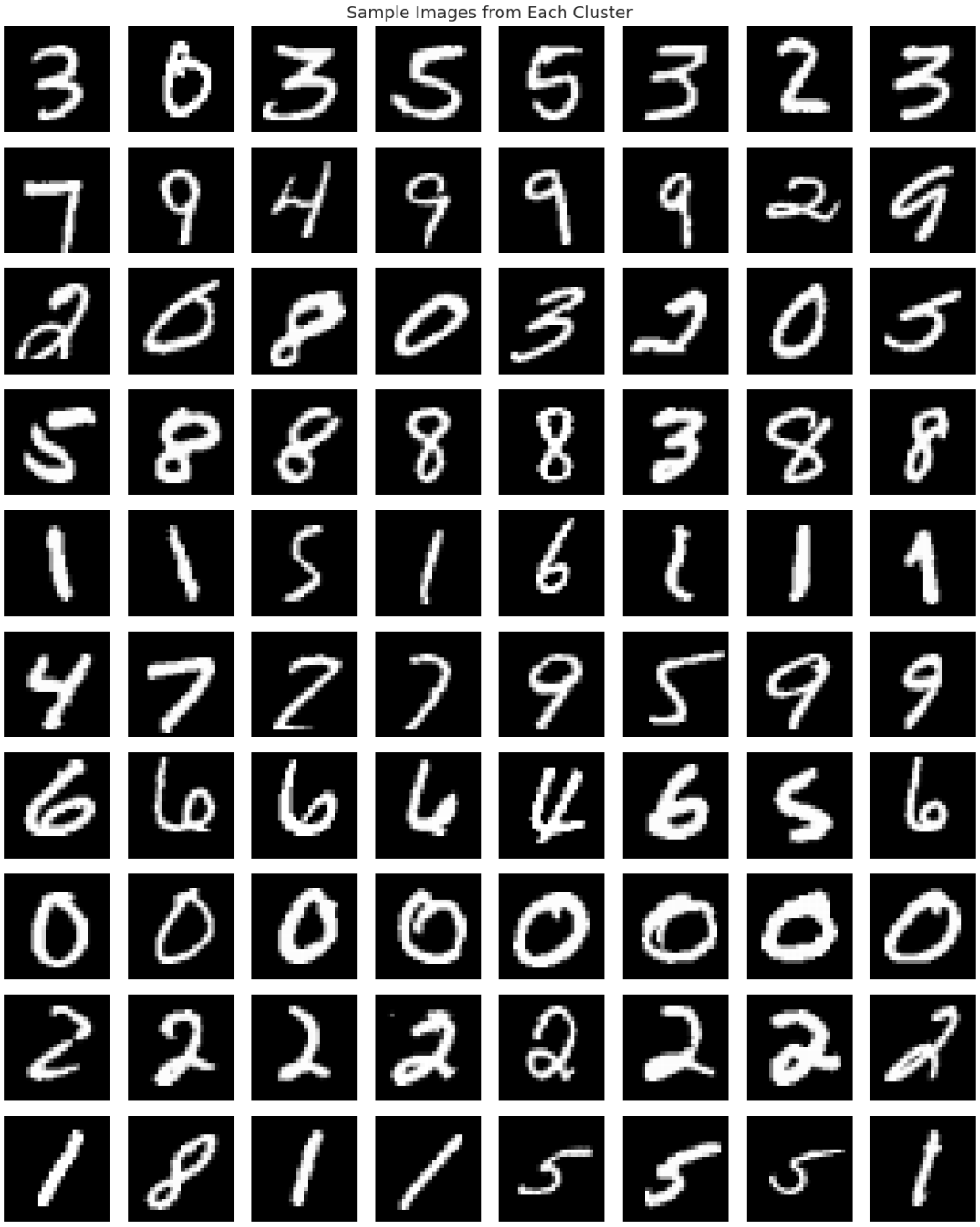

Figure 9: Eight sample images drawn from each of the 10 clusters

Inference:

Looking at actual sample images per cluster rather than just the averaged centroid confirms the same story – most rows are dominated by one digit shape with just the occasional stray image that looks like it wandered in from a visually similar digit. This kind of "mostly right, occasionally confused" pattern is typical of K-Means on raw-ish pixel data; a more sophisticated method (like clustering on a learned embedding instead of PCA) would likely tighten this up further.

11 Discussion

This experiment showed that PCA and K-Means can discover meaningful structure even without class labels. Reducing the data from 784 dimensions to 100 preserved most of the important information while making clustering computationally efficient. Although the silhouette score was relatively low, the cluster centroids and t-SNE visualization indicated that several handwritten digits formed recognizable groups. The main limitation was that visually similar digits were sometimes grouped together, suggesting that more advanced clustering methods such as DBSCAN or non-linear dimensionality reduction techniques could further improve the results.

12 Conclusion

PCA combined with K-Means successfully clustered the unlabeled MNIST dataset into meaningful groups. Dimensionality reduction significantly reduced computational cost while preserving the essential data structure. The experiment demonstrates that unsupervised learning can reveal useful patterns even without class labels, though cluster quality could be further improved using more advanced feature extraction and clustering techniques.

13 References

1. Chitraju Vishnu Vineeth, "ML_LAB_ASSIGNMENTS – Assignment 1," GitHub repository. [Online]. Available: https://github.com/VishnuVineeth14/ML_LAB_ASSIGNMENTS/tree/main/Assignment1. [Accessed: 27-Jul-2026].